

- vader_polarity() 함수 생성
 - 입력 파라미터로 영화 감성평 텍스트와 긍정/부정 결정하는 임계값(threshold) 가짐
 - SentimentIntensityAnalyzer 객체의 polarity_scores() 메서드 호출해 감성 결과 반환
- 각 문서별로 감성 결과를 vader_preds 라는 review_df 의 새로운 칼럼으로 저장
- 저장된 감성 분석 결과 기반으로 VADER 예측 성능 측정
 - => 정확도 SentiWordNet보다 향상, 재현율 매우 크게 향상

(6) 토픽 모델링 - 20 뉴스그룹

토픽 모델링 (Topic Modeling)

- 문서 집합에 숨어 있는 주제 찾아내는 것
- ML 기반의 토픽 모델은 숨겨진 주제 효과적으로 표현할 수 있는 중심 단어 함축적으로 추출
- 자주 사용되는 기법 => LSA (Latent Semantic Analysis) & LDA (Latent Dirichlet Allocation)
- LatentDirichletAllocation : 사이킷런의 LDA 기반 토픽 모델링 클래스

<20 뉴스그룹 데이터셋>

```
['alt.atheism', 'comp.graphics', 'comp.os.ms-windows.misc', 'comp.sys.ibm.pc.hardware',
'comp.sys.mac.hardware', 'comp.windows.x', 'misc.forsale', 'rec.autos', 'rec.motorcycles',
'rec.sport.baseball', 'rec.sport.hockey', 'sci.crypt', 'sci.electronics', 'sci.med', 'sci.space',
'soc.religion.christian', 'talk.politics.guns', 'talk.politics.mideast', 'talk.politics.misc',
'talk.religion.misc']
```

모터사이클, 야구, 그래픽스, 윈도우, 중동, 기독교, 전자공학, 의학 8개 주제 추출하고 텍스트에 LDA 기반 토픽 모델링 적용

```

from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

# 모터사이클, 야구, 그래픽스, 윈도우즈, 중동, 기독교, 의학, 우주 주제를 추출.
cats = ['rec.motorcycles', 'rec.sport.baseball', 'comp.graphics', 'comp.windows.x',
        'talk.politics.mideast', 'soc.religion.christian', 'sci.electronics', 'sci.med' ]

# 위에서 cats 변수로 기재된 category만 추출. fetch_20newsgroups( )의 categories에 cats 입력
news_df = fetch_20newsgroups(subset='all', remove=('headers', 'footers', 'quotes'),
                             categories=cats, random_state=0)

#LDA는 Count 기반의 Vectorizer만 적용합니다.
count_vect = CountVectorizer(max_df=0.95, max_features=1000, min_df=2, stop_words='english', ngram_range=(1,2))
feat_vect = count_vect.fit_transform(news_df.data)
print('CountVectorizer Shape:', feat_vect.shape)

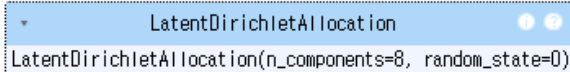
```

CountVectorizer Shape: (7862, 1000)

```

lda = LatentDirichletAllocation(n_components=8, random_state=0)
lda.fit(feat_vect)

```



```

print(lda.components_.shape)
lda.components_

(8, 1000)
array([[2.69030238e+02, 1.87798026e+02, 7.09003824e+01, ...,
        1.22710343e+01, 1.06329639e+02, 7.25995512e+01],
       [1.25091799e-01, 2.46049106e+00, 1.25051902e-01, ...,
        2.80071176e+02, 1.25089783e-01, 5.05669662e+01],
       [1.33978420e+02, 1.25042012e-01, 9.98277256e+01, ...,
        1.25092219e-01, 3.31078261e+01, 1.25026398e-01],
       ...,
       [2.98813886e+01, 1.88071366e+01, 1.14748730e+01, ...,
        1.93022584e+01, 5.29368271e+00, 1.44478198e+01],
       [1.25074899e-01, 1.25105300e-01, 1.25004235e-01, ...,
        1.03576436e+02, 1.25100535e-01, 7.22276359e+01],
       [1.25172284e-01, 1.03967760e+00, 1.25221075e-01, ...,
        5.31740996e+01, 1.25025929e-01, 1.25062991e-01]])

```

- fetch_20newsgroups() API => categories 파라미터 통해 필요한 주제만 필터링해 추출
- 추출한 텍스트 Count 기반으로 벡터화 변환
 - max_features=1000 으로 word 피쳐 개수 제한
 - ngram_range 는 (1,2)로 설정
- 피쳐 벡터화된 데이터셋 기반으로 LDA 토픽 모델링 수행
 - 토픽 개수는 위와 동일한 8개
 - LatentDirichletAllocation 클래스의 n_components 파라미터로 토픽 개수 조정
 - random_state 값은 예제 수행할 때마다 결과 똑같이 하기 위해 입력
- LatentDirichletAllocation 객체의 components_ 형태와 속성값 확인
 - components 는 개별 토픽별로 각 word 피쳐가 얼마나 많이 그 토픽에 할당됐는지에 대한 수치 가짐
 - 높은 값일수록 토픽의 중심 word
 - array[8,1000] => 8개의 토픽별로 1000개의 word 피쳐가 해당 토픽별 연관도 값 가짐

- components_array 의 0번째 row, 10번째 col에 있는 값은 Topic #0에 대해 피쳐 벡터화된 행렬에서 10번째 칼럼에 해당하는 피쳐가 Topic #0에 연관되는 수치 값 가짐

```
def display_topics(model, feature_names, no_top_words):
    for topic_index, topic in enumerate(model.components_):
        print('Topic #', topic_index)

        # components_array에서 가장 값이 큰 순으로 정렬했을 때, 그 값의 array index를 반환.
        topic_word_indexes = topic.argsort()[::-1]
        top_indexes = topic_word_indexes[:no_top_words]

        # top_indexes대상인 index별로 feature_names에 해당하는 word feature 추출 후 join으로 concat
        feature_concat = ' '.join([feature_names[i] for i in top_indexes])
        print(feature_concat)

# CountVectorizer객체내의 전체 word들의 명칭을 추출
feature_names = count_vect.get_feature_names_out()

# 토픽별 가장 연관도가 높은 word를 15개만 추출
display_topics(lda, feature_names, 15)
```

Topic # 0
10 year medical health 1993 20 12 disease cancer team patients research number new 11

Topic # 1
don just like know think good time ve does way really people want 11 right

Topic # 2
image file jpeg output program gif images format files color entry use bit 03 02

Topic # 3
armenian armenians turkish people said turkey armenia government genocide turks muslim russian greek azerbaijan killed

Topic # 4
israel jews dos jewish israeli dos dos arab state people arabs palestinian adl ed anti peace

Topic # 5
edu com available graphics ftp window use mail data motif software version pub information server

Topic # 6
god people jesus church believe say christ does christian think christians did know bible man

Topic # 7
thanks use using does help like display need problem know server screen windows window program

- display_topics() 함수 만들어 각 토픽별 연관도 높은 순으로 word 나열

(7) 문서 군집화 소개와 실습

문서 군집화

- 비슷한 텍스트 구성의 문서를 군집화하는 것
- 동일한 군집에 속하는 문서를 같은 카테고리 소속으로 분류할 수 있다는 점에서 텍스트 분류 기반 문서 분류와 유사
- BUT 사전에 결정 카테고리 값 가진 학습 데이터셋 필요한 텍스트 분류 기반 문서 분류와 달리, 문서 군집화는 학습 데이터셋 필요 없는 비지도학습 기반으로 동작

<Opinion Review 데이터셋 이용한 문서 군집화 구행하기>

```

import pandas as pd
import glob, os
import warnings
warnings.filterwarnings('ignore')
pd.set_option('display.max_colwidth', 700)

path = '/content/OpinosisDataset1.0/topics'

# path로 지정한 디렉터리 밑에 있는 모든 .data 파일들의 파일명을 리스트로 취합
all_files = glob.glob(os.path.join(path, "*.data"))
filename_list = []
opinion_text = []

# 개별 파일들의 파일명은 filename_list 리스트로 취합,
# 개별 파일들의 파일 내용은 DataFrame 로딩 후 다시 string으로 변환하여 opinion_text 리스트로 취합
for file_ in all_files:
    # 개별 파일을 읽어서 DataFrame으로 생성
    df = pd.read_table(file_, index_col=None, header=0, encoding='latin1')

    # 절대경로로 주어진 file 명을 가공. 만일 Linux에서 수행시에는 아래 ㄱ를 / 변경.
    # 맨 마지막 .data 확장자도 제거
    filename_ = file_.split('ㄱ')[-1]
    filename = filename_.split('.')[0]

    # 파일명 리스트와 파일 내용 리스트에 파일명과 파일 내용을 추가.
    filename_list.append(filename)
    opinion_text.append(df.to_string())

# 파일명 리스트와 파일 내용 리스트를 DataFrame으로 생성
document_df = pd.DataFrame({'filename':filename_list, 'opinion_text':opinion_text})
document_df.head()

```

	filename	opinion_text
0	/content/OpinosisDataset1	Keep in mind that once you get in a room full of light or step outdoors screen reflections could become annoying .\n0 I've used mine outsi...
1	/content/OpinosisDataset1	You also get upscale features like spoken directions including street names and programmable POIs .\n0 I used to hesitate to go out of my directions but no...
2	/content/OpinosisDataset1	I thought it would be fitting to christen my Kindle with the Stephen King novella UR, so went to the Amazon site on my computer and clicked on the button to buy it .\n0 As soon as I'd clicked the button to confirm my order it appeared on my Kindle almost immediately !\n1 ...
3	/content/OpinosisDataset1	Good Value good location , ideal choice .\n0 Great Location , Nice Rooms , Helpless Concierge\n1 ...
4	/content/OpinosisDataset1	After slowing down, transmission has to be kicked to speed up .\n0 ...

- for 반복문으로 개별 파일명을 파일명 리스트에 추가
- DataFrame으로 개별 파일 읽은 후 다시 문자열로 반환해 파일 내용 리스트에 추가
- 만들어진 파일명 및 파일 내용 리스트 이용해 새롭게 파일명과 파일 내용 칼럼으로 갖는 DataFrame 생성

```

from nltk.stem import WordNetLemmatizer
import nltk
import string

remove_punct_dict = dict((ord(punct), None) for punct in string.punctuation)
lemmar = WordNetLemmatizer()

# 입력으로 들어온 token단어들에 대해서 lemmatization 머진 변환.
def LemTokens(tokens):
    return [lemmar.lemmatize(token) for token in tokens]

# TfidfVectorizer 객체 생성 시 tokenizer인자로 해당 함수를 설정하여 lemmatization 적용
# 입력으로 문장을 받아서 stop words 제거-> 소문자 변환 -> 단어 토큰화 -> lemmatization 머진 변환.
def LemNormalize(text):
    return LemTokens(nltk.word_tokenize(text.lower().translate(remove_punct_dict)))

```

```
import nltk
```

```

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('omw-1.4')

```

```

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
True

```

```

from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vect = TfidfVectorizer(tokenizer=LemNormalize, stop_words='english', #
                             ngram_range=(1,2), min_df=0.05, max_df=0.85 )

#opinion_text 컬럼값으로 feature vectorization 수행
feature_vect = tfidf_vect.fit_transform(document_df['opinion_text'])

```

```

from sklearn.cluster import KMeans

# 5개 집합으로 군집화 수행. 예제를 위해 동일한 클러스터링 결과 도출용 random_state=0
km_cluster = KMeans(n_clusters=5, max_iter=10000, random_state=0)
km_cluster.fit(feature_vect)
cluster_label = km_cluster.labels_
cluster_centers = km_cluster.cluster_centers_

```

```

document_df['cluster_label'] = cluster_label
document_df.head()

```

	filename	opinion_text	cluster_label
0	/content/OpinosisDataset1	Keep in mind that once you get in a room full of light or step outdoors screen reflections could become annoying .\n0 I've used mine outsi...	2
1	/content/OpinosisDataset1	You also get upscale features like spoken directions including street names and programmable POIs .\n0 I used to hesitate to go out of my directions but no...	3
2	/content/OpinosisDataset1	I thought it would be fitting to christen my Kindle with the Stephen King novella UR, so went to the Amazon site on my computer and clicked on the button to buy it .\n0 As soon as I'd clicked the button to confirm my order it appeared on my Kindle almost immediately !\n1 ...	4
3	/content/OpinosisDataset1	Good Value good location , ideal choice .\n0 Great Location , Nice Rooms , Helpless Concierge\n1 ...	0
4	/content/OpinosisDataset1	After slowing down, transmission has to be kicked to speed up .\n0 ...	1

- 문서를 TF-IDF 형태로 피쳐 벡터화
 - LemNormarlize() 함수 이용
 - ngram 은 (1,2)로, min_df 와 max_df 범위 설정해 피쳐 개수 제한
 - fit_transform() 인자로 document_df DataFrame의 opinion_text 칼럼 입력
- 문서별 텍스트가 TF-IDF 변환된 피쳐 벡터화 행렬 데이터에 대해 군집화 수행해 어떤 문서끼리 군집되는지 확인
 - 군집화 기법으로 K-평균 적용
 - 먼저 5개의 중심 기반으로 어떻게 군집화되는지 확인
 - max_iter 는 10000으로 설정
 - KMeans 수행 후 군집의 Label 값과 중심별로 할당된 데이터셋의 좌표 값 구함
- 각 데이터별로 할당된 군집의테이블을 document_df DataFrame에 cluster_label 칼럼 추가해 저장

```
document_df[document_df['cluster_label']==0].sort_values(by='filename')
```

```
document_df[document_df['cluster_label']==1].sort_values(by='filename')
```

```
document_df[document_df['cluster_label']==2].sort_values(by='filename')
```

```
document_df[document_df['cluster_label']==3].sort_values(by='filename')
```

```
document_df[document_df['cluster_label']==4].sort_values(by='filename')
```

- 정렬칼럼명 cluster_label 값에 따른 군집화 결과 확인

```

from sklearn.cluster import KMeans

# 3개의 집합으로 군집화
km_cluster = KMeans(n_clusters=3, max_iter=10000, random_state=0)
km_cluster.fit(feature_vect)
cluster_label = km_cluster.labels_

# 소속 클러스터를 cluster_label 컬럼으로 할당하고 cluster_label 값으로 정렬
document_df['cluster_label'] = cluster_label
document_df.sort_values(by='cluster_label')

```

	filename	opinion_text	cluster_label
3	/content/OpinosisDataset1	Good Value good location , ideal choice .\n0 Great Location , Nice Rooms , Helpless Concierge\n1 ...	0
6	/content/OpinosisDataset1	Both of us having worked in tourism for over 14 years were very disappointed at the level of service provided by this gentleman .\n0 The service was good, very friendly staff and we loved the free wine reception each night .\n1 ...	0
9	/content/OpinosisDataset1	not customer, oriented hotelvery low service levelboor reception\n0 The room was quiet, clean, the bed and pillows were comfortable, and the serv...	0
8	/content/OpinosisDataset1	The food for our event was delicious .\n0 ...	0
15	/content/OpinosisDataset1	The room was packed to capacity with queues at the food buffets .\n0 The over zealous staff cleared our unfinished drinks while we were collecting cooked food and movement around the room with plates was difficult in the crowded circumstances .\n1 ...	0
27	/content/OpinosisDataset1	All in all, a normal chain hotel on a nice location , I will be back if I do not find anthing closer to Picadilly for a better price .\n0 ...	0
19	/content/OpinosisDataset1	The Swissotel is one of our favorite hotels in Chicago and the corner rooms have the most fantastic views in the city .\n0 The rooms look like they were just remodled and upgraded, there was an HD TV and a nice iHome docking station to put my iPod so I could set the alarm to wake up with my music instead of the radio .\n1 ...	0
17	/content/OpinosisDataset1	Mediocre room and service for a very extravagant price .\n0 ...	0
24	/content/OpinosisDataset1	We arrived at 23,30 hours and they could not recommend a restaurant so we decided to go to Tesco, with very limited choices but when you are hingry you do not careNext day they rang the bell at 8,00 hours to clean the room, not being very nice being waken up so earlyEvery day they gave u...	0

- 중심 개수를 5개에서 3개로 낮춰 3개 그룹으로 군집화한 결과 확인

<군집별 핵심 단어 추출하기>

```
cluster_centers = km_cluster.cluster_centers_
print('cluster_centers shape : ', cluster_centers.shape)
print(cluster_centers)
```

```
cluster_centers shape : (3, 4610)
[[0.          0.00093692 0.00164382 ... 0.          0.00172609 0.00136077]
 [0.00170594 0.00057845 0.          ... 0.00514075 0.          0.          ]
 [0.01244819 0.          0.          ... 0.00523999 0.          0.          ]]
```

```
# 군집별 top n 핵심단어, 그 단어의 중심 위치 상대값, 대상 파일명들을 반환함.
def get_cluster_details(cluster_model, cluster_data, feature_names, clusters_num, top_n_features=10):
    cluster_details = {}

    # cluster_centers array 의 값이 큰 순으로 정렬된 index 값을 반환
    # 군집 중심점(centroid)별 할당된 word 피쳐들의 거리값이 큰 순으로 값을 구하기 위함.
    centroid_feature_ordered_ind = cluster_model.cluster_centers_.argsort()[::-1]

    # 개별 군집별로 iteration하면서 핵심단어, 그 단어의 중심 위치 상대값, 대상 파일명 입력
    for cluster_num in range(clusters_num):
        # 개별 군집별 정보를 담은 데이터 초기화.
        cluster_details[cluster_num] = {}
        cluster_details[cluster_num]['cluster'] = cluster_num

        # cluster_centers_.argsort()[::-1] 로 구한 index 를 이용하여 top n 피쳐 단어를 구함.
        top_feature_indexes = centroid_feature_ordered_ind[cluster_num, :top_n_features]
        top_features = [ feature_names[ind] for ind in top_feature_indexes ]

        # top_feature_indexes를 이용해 해당 피쳐 단어의 중심 위치 상대값 구함
        top_feature_values = cluster_model.cluster_centers_[cluster_num, top_feature_indexes].tolist()

        # cluster_details 딕셔너리 객체에 개별 군집별 핵심 단어와 중심위치 상대값, 그리고 해당 파일명 입력
        cluster_details[cluster_num]['top_features'] = top_features
        cluster_details[cluster_num]['top_features_value'] = top_feature_values
        filenames = cluster_data[cluster_data['cluster_label'] == cluster_num]['filename']
        filenames = filenames.values.tolist()
        cluster_details[cluster_num]['filenames'] = filenames

    return cluster_details
```

- 앞에서 군집 3개로 생성한 KMeans 객체인 km_cluster 에서 cluster_centers_ 속성값 가져와 확인
 - (3,4610) 배열 => 군집이 3개, word 피쳐가 4610개로 구성
 - 각 행의 배열 값은 각 군집 내의 4610개 피쳐의 위치가 개별 중심과 얼마나 가까운가 상대 값으로 나타낸 것 (0에서 1까지 값 가짐, 1에 가까울수록 중심과 가까운 값 의미)
- cluster_centers_ 속성인 넘파이 ndarray의 argsort()[::-1] 이용해 각 군집별 핵심 단어 찾기
 - cluster_centers 배열 내 값이 큰 순으로 정렬된 위치 인덱스 값 반환
 - 핵심 단어 피쳐 이름 출력 가능
 - get_cluster_details() 함수 => 배열 내에서 값 가장 큰 데이터의 위치 인덱스 추출한 뒤, 위치 인덱스 이용해 핵심 단어 이름과 상대 위치 값 추출해 cluster_details 라는 Dict 객체 변수에 기록하고 반환


```
def print_cluster_details(cluster_details):
    for cluster_num, cluster_detail in cluster_details.items():
        print('##### Cluster {}'.format(cluster_num))
        print('Top features:', cluster_detail['top_features'])
        print('Reviews 파일명 :', cluster_detail['filenames'][:?])
        print('=====')
```

```
feature_names = tfidf_vect.get_feature_names_out()
```

```
cluster_details = get_cluster_details(cluster_model=km_cluster, cluster_data=document_df, #
                                     feature_names=feature_names, clusters_num=3, top_n_features=10 )
print_cluster_details(cluster_details)
```

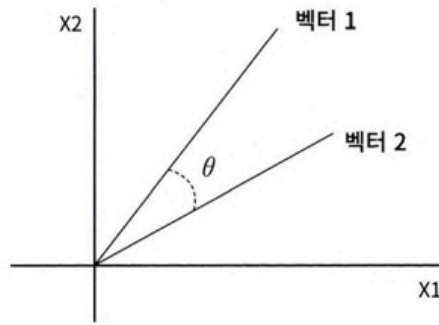
```
##### Cluster 0
Top features: ['room', 'hotel', 'service', 'staff', 'food', 'location', 'price', 'bathroom', 'clean', 'parking']
Reviews 파일명 : ['/content/OpinosisDataset1', '/content/OpinosisDataset1', '/content/OpinosisDataset1',
=====
##### Cluster 1
Top features: ['interior', 'seat', 'mileage', 'comfortable', 'gas', 'gas mileage', 'car', 'transmission', 'performance', 'voice']
Reviews 파일명 : ['/content/OpinosisDataset1', '/content/OpinosisDataset1', '/content/OpinosisDataset1', '/content/OpinosisDataset1',
=====
##### Cluster 2
Top features: ['screen', 'battery', 'keyboard', 'battery life', 'life', 'video', 'direction', 'button', 'size', 'map']
Reviews 파일명 : ['/content/OpinosisDataset1', '/content/OpinosisDataset1', '/content/OpinosisDataset1', '/content/OpinosisDataset1',
=====
```

- print_cluster_details() 함수 => cluster_details 의 속성 값 정보 더 보기 좋게 표현
- get_cluster_details(), print_cluster_details() 호출

(8) 문서 유사도

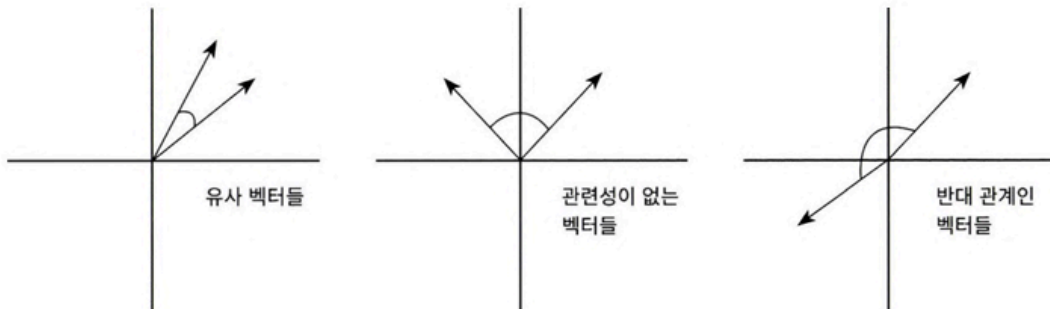
문서 유사도 측정 방법 - 코사인 유사도

- 벡터와 벡터 간 유사도 비교할 때 벡터의 크기보다는 벡터의 상호 방향성이 얼마나 유사한지에 기반
- 두 벡터 사이의 사잇각 구해 얼마나 유사한지 수치로 적용한 것



두 벡터 사잇각

두 벡터의 사잇각에 따라서 상호 관계는 다음과 같이 유사하거나 관련이 없거나 아예 반대 관계가 될 수 있습니다.



따라서 유사도 $\cos\theta$ 는 다음과 같이 두 벡터의 내적을 총 벡터 크기의 합으로 나눈 것입니다(즉, 내적 결과를 총 벡터 크기로 정규화(L2 Norm)한 것입니다).

$$\text{similarity} = \cos\theta = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

- 코사인 유사도가 문서 유사도 비교에 가장 많이 사용되는 이유
 - 문서를 피처 벡터화 변환 시 매우 많은 희소 행렬 되기 쉽고, 이러한 희소 행렬 기반에서 문서와 문서 벡터 간 크기에 기반한 유사도 지표 정확도 떨어지기 쉬움
 - 문서 매우 긴 경우 단어 빈도수도 더 많을 것이므로 빈도수에만 기반해서는 공정한 비교 X

<간단 예시>

```
import sklearn
import numpy as np

def cos_similarity(v1, v2):
    dot_product = np.dot(v1, v2)
    l2_norm = (np.sqrt(sum(np.square(v1))) * np.sqrt(sum(np.square(v2))))
    similarity = dot_product / l2_norm

    return similarity
```

```
from sklearn.feature_extraction.text import TfidfVectorizer

doc_list = ['if you take the blue pill, the story ends' ,
            'if you take the red pill, you stay in Wonderland',
            'if you take the red pill, I show you how deep the rabbit hole goes']

tfidf_vect_simple = TfidfVectorizer()
feature_vect_simple = tfidf_vect_simple.fit_transform(doc_list)
print(feature_vect_simple.shape)
```

```
(3, 18)
```

```
# TfidfVectorizer로 transform()한 결과는 Sparse Matrix이므로 Dense Matrix로 변환.
feature_vect_dense = feature_vect_simple.todense()

#첫번째 문장과 두번째 문장의 feature vector 추출
vect1 = np.array(feature_vect_dense[0]).reshape(-1,)
vect2 = np.array(feature_vect_dense[1]).reshape(-1,)

#첫번째 문장과 두번째 문장의 feature vector로 두개 문장의 Cosine 유사도 추출
similarity_simple = cos_similarity(vect1, vect2)
print('문장 1, 문장 2 Cosine 유사도: {0:.3f}'.format(similarity_simple))
```

```
문장 1, 문장 2 Cosine 유사도: 0.402
```

- `cos_similarity()` => 두 개 넘파이 배열에 대한 코사인 유사도 구하는 함수
- `doc_list` 로 정의된 3개의 간단한 문서 유사도 비교하기 위해 이 문서를 TF-IDF로 벡터화된 행렬로 변환
- 변환된 행렬은 희소 행렬이므로 array로 만들기 위해 밀집 행렬로 변환한 뒤 다시 각각 배열로 변환
- `cos_similarity()` 함수 이용해 두 개 문서의 유사도 측정

```
vect1 = np.array(feature_vect_dense[0]).reshape(-1,)
vect3 = np.array(feature_vect_dense[2]).reshape(-1,)
similarity_simple = cos_similarity(vect1, vect3)
print('문장 1, 문장 3 Cosine 유사도: {0:.3f}'.format(similarity_simple))

vect2 = np.array(feature_vect_dense[1]).reshape(-1,)
vect3 = np.array(feature_vect_dense[2]).reshape(-1,)
similarity_simple = cos_similarity(vect2, vect3)
print('문장 2, 문장 3 Cosine 유사도: {0:.3f}'.format(similarity_simple))
```

```
문장 1, 문장 3 Cosine 유사도: 0.404
문장 2, 문장 3 Cosine 유사도: 0.456
```

```
from sklearn.metrics.pairwise import cosine_similarity

similarity_simple_pair = cosine_similarity(feature_vect_simple[0] , feature_vect_simple)
print(similarity_simple_pair)
```

```
[[1.          0.40207758  0.40425045]]
```

```
from sklearn.metrics.pairwise import cosine_similarity

similarity_simple_pair = cosine_similarity(feature_vect_simple[0] , feature_vect_simple[1:])
print(similarity_simple_pair)
```

```
[[0.40207758  0.40425045]]
```

```
similarity_simple_pair = cosine_similarity(feature_vect_simple , feature_vect_simple)
print(similarity_simple_pair)
print('shape:',similarity_simple_pair.shape)
```

```
[[1.          0.40207758  0.40425045]
 [0.40207758  1.          0.45647296]
 [0.40425045  0.45647296  1.          ]]
shape: (3, 3)
```

- 첫 번째 문장과 세 번째 문장, 두 번째 문장과 세 번째 문장의 유사도 역시 측정
- 사이킷런 API `sklearn.metrics.pairwise.cosine_similarity` 이용해 문서 유사도 측정
 - 첫 번째 파라미터 => 비교 기준 되는 문서의 피처 행렬
 - 두 번째 파라미터 => 비교되는 문서의 피처 행렬
 - `cosine_similarity()` 는 희소 행렬과 밀집 행렬, 행렬 또는 배열 모두 가능하므로 별도의 변환 작업 필요 X
- 비교 대상에서 `feature_vect[1:]` 이용해 비교 기준 문서 제외
- 모든 개별 문서에 쌍으로 코사인 유사도 값 계산

<Opinion Review 데이터셋 이용한 문서 유사도 측정>

```
import pandas as pd
import glob, os
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
import warnings
warnings.filterwarnings('ignore')

path = '/content/OpinosisDataset1.0/topics'
all_files = glob.glob('/content/OpinosisDataset1.0/topics/**/*.data', recursive=True)
filename_list = []
opinion_text = []

for file_ in all_files:
    df = pd.read_table(file_, index_col=None, header=0, encoding='latin1')
    filename = os.path.splitext(os.path.basename(file_))[0]
    filename_list.append(filename)
    opinion_text.append(df.to_string())
```

```
document_df = pd.DataFrame({'filename':filename_list, 'opinion_text':opinion_text})

tfidf_vect = TfidfVectorizer(tokenizer=LemNormalize, stop_words='english' , \
                             ngram_range=(1,2), min_df=0.05, max_df=0.85 )
feature_vect = tfidf_vect.fit_transform(document_df['opinion_text'])

km_cluster = KMeans(n_clusters=3, max_iter=10000, random_state=0)
km_cluster.fit(feature_vect)
cluster_label = km_cluster.labels_
cluster_centers = km_cluster.cluster_centers_
document_df['cluster_label'] = cluster_label
```

```
from sklearn.metrics.pairwise import cosine_similarity

# DataFrame에서 호텔로 클러스터링된 데이터의 Index를 추출
comparison_idx = document_df[
    document_df['filename'] == 'bathroom_bestwestern_hotel_sfo.txt'
].index[0]

# 호텔로 클러스터링된 데이터 중 첫번째 문서를 추출하여 파일명 표시.
comparison_docname = document_df.loc[comparison_idx]['filename']
print('#### 비교 기준 문서명 ',comparison_docname,' 와 타 문서 유사도####')

''' document_df에서 추출한 Index 객체를 feature_vect로 입력하여 호텔 클러스터링된 feature_vect 추출
이를 이용하여 호텔로 클러스터링된 문서 중 첫번째 문서와 다른 문서간의 코사인 유사도 측정.'''
similarity_pair = cosine_similarity(feature_vect[hotel_indexes[0]] , feature_vect[hotel_indexes])
print(similarity_pair)

#### 비교 기준 문서명 bathroom_bestwestern_hotel_sfo.txt 와 타 문서 유사도####
[[1.         0.26210568 0.08674216 0.23224038 0.1102953  0.1903599
  0.22075729 0.16748876 0.31010708 0.12023457 0.24011893 0.09219582
  0.05857476 0.2612438  0.17364444 0.0215757  0.66967861]]
```

- 데이터셋 DataFrame으로 로드 후 호텔 주제로 군집화된 데이터 먼저 추출하고 이 데이터에 해당하는 TfidfVectorizer 의 데이터 추출
 - 데이터셋 객체 변수 document_df 에서 먼저 호텔로 군집화된 문서의 인덱스 추출
 - 추출된 인덱스 이용해 TfidfVectorizer 객체 변수인 feature_vect 에서 호텔로 군집화된 문서의 피쳐 벡터 추출

```

import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

# argsort()를 이용하여 앞예제의 첫번째 문서와 타 문서간 유사도가 큰 순으로 정렬한 인덱스 반환하되 자기 자신은 제외.
sorted_index = similarity_pair.argsort()[::-1]
sorted_index = sorted_index[:, 1:]

# 유사도가 큰 순으로 hotel_indexes를 추출하여 재 정렬.
hotel_sorted_indexes = hotel_indexes[sorted_index.reshape(-1)]

# 유사도가 큰 순으로 유사도 값을 재정렬하되 자기 자신은 제외
hotel_l_sim_value = np.sort(similarity_pair.reshape(-1))[::-1]
hotel_l_sim_value = hotel_l_sim_value[1:]

# 유사도가 큰 순으로 정렬된 Index와 유사도값을 이용하여 파일명과 유사도값을 Seaborn 막대 그래프로 시각화
hotel_l_sim_df = pd.DataFrame()
hotel_l_sim_df['filename'] = document_df.iloc[hotel_sorted_indexes]['filename']
hotel_l_sim_df['similarity'] = hotel_l_sim_value
print('가장 유사도가 큰 파일명 및 유사도:\n', hotel_l_sim_df.iloc[0,:])

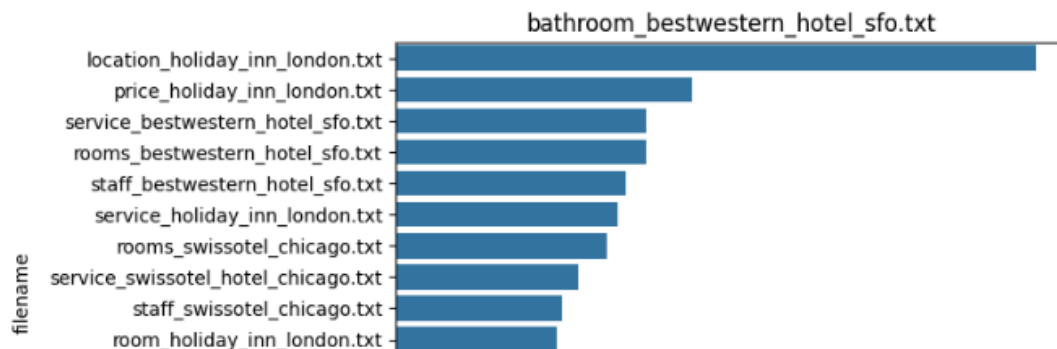
fig1 = plt.gcf()
sns.barplot(x='similarity', y='filename', data=hotel_l_sim_df)
plt.title(comparison_docname)
fig1.savefig('p553_hotel.tif', format='tif', dpi=300, bbox_inches='tight')

```

```

... 가장 유사도가 큰 파일명 및 유사도:
filename      location_holiday_inn_london.txt
similarity                                0.669679
Name: 49, dtype: object

```



- 첫 번째 문서와 다른 문서 간 유사도 높은 순으로 정렬하고 시각화
 - cosine_similarity() 는 쌍 형태의 ndarray 반환하므로 판다스 인덱스로 이용하기 위해 reshape(-1) 로 차원 변경

(9) 한글 텍스트 처리 - 네이버 영화 평점 감성 분석

한글 NLP 처리의 어려움

- '띄어쓰기'와 '다양한 조사' 때문에 일반적으로 한글 처리는 영어 등의 라틴어 처리보다 어려움
- 띄어쓰기 잘못하면 의미 왜곡돼 전달될 수 있으며, 조사는 경우의 수가 많아 어근 추출 등 전처리 시 제거하기 까다로움

KoNLPy

- 파이썬의 대표적인 한글 형태소 패키지
- 설치 시 공식 설치 문서 및 교재 p.559 ~ 560 참고

<데이터 로딩>

```
import pandas as pd

train_df = pd.read_csv('ratings_train.txt', sep='\\t')
train_df.head(3)
```

	id	document	label
0	9976970	아 더빙.. 진짜 짜증나네요 목소리	0
1	3819312	흠...포스터보고 초딩영화줄....오버연기조차 가볍지 않구나	1
2	10265843	너무재밌었다그래서보는것을추천한다	0

```
train_df['label'].value_counts()
```

	count
label	
0	75173
1	74827

```
dtype: int64
```

```
import re

train_df = train_df.fillna(' ')
# 정규 표현식을 이용하여 숫자를 공백으로 변경(정규 표현식으로 \d 는 숫자를 의미함.)
train_df['document'] = train_df['document'].apply( lambda x : re.sub(r"\d+", " ", x) )

# 테스트 데이터 셋을 로딩하고 동일하게 Null 및 숫자를 공백으로 변환
test_df = pd.read_csv('ratings_test.txt', sep='\\t')
test_df = test_df.fillna(' ')
test_df['document'] = test_df['document'].apply( lambda x : re.sub(r"\d+", " ", x) )

# id 칼럼 삭제 수행
train_df.drop('id', axis=1, inplace=True)
test_df.drop('id', axis=1, inplace=True)
```

- ratings_train.txt 파일 DataFrame으로 로드
- 학습 데이터셋의 0과 1의 Label 값 비율 살피기 (1이 긍정, 0이 부정 감성) => 균등한 분포 확인
- document 칼럼에 일부 존재하는 Null 정규 표현식 모듈 re 이용해 공백으로 변환

```
from konlpy.tag import Twitter

twitter = Twitter()
def tw_tokenizer(text):
    # 입력 문자로 들어온 text 를 형태소 단어로 토큰화 하며 list 객체 반환
    tokens_ko = twitter.morphs(text)
    return tokens_ko
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Twitter 객체의 morphs( ) 객체를 이용한 tokenizer를 사용. ngram_range는 (1,2)
tfidf_vect = TfidfVectorizer(tokenizer=tw_tokenizer, ngram_range=(1,2), min_df=3, max_df=0.9)
tfidf_vect.fit(train_df['document'])
tfidf_matrix_train = tfidf_vect.transform(train_df['document'])
```

```
# 로지스틱 회귀를 이용해 감성 분석 분류 수행
lg_clf = LogisticRegression(random_state=0, solver='liblinear')

# Parameter C 최적화를 위해 GridSearchCV 를 이용
params = { 'C': [1, 3.5, 4.5, 5.5, 10] }
grid_cv = GridSearchCV(lg_clf, param_grid=params, cv=3, scoring='accuracy', verbose=1)
grid_cv.fit(tfidf_matrix_train, train_df['label'])
print(grid_cv.best_params_, round(grid_cv.best_score_,4))
```

```
Fitting 3 folds for each of 5 candidates, totalling 15 fits
{'C': 3.5} 0.8593
```

```
from sklearn.metrics import accuracy_score

# 학습 데이터를 적용한 TfidfVectorizer를 이용하여 테스트 데이터를 TF-IDF 값으로 Feature 변환함.
tfidf_matrix_test = tfidf_vect.transform(test_df['document'])

# classifier 는 GridSearchCV에서 최적 파라미터로 학습된 classifier를 그대로 이용
best_estimator = grid_cv.best_estimator_
preds = best_estimator.predict(tfidf_matrix_test)

print('Logistic Regression 정확도: ', accuracy_score(test_df['label'], preds))
```

```
Logistic Regression 정확도: 0.86172
```

- 각 문장을 한글 형태소 분석 통해 형태소 단어로 토큰화
 - 한글 형태소 엔진은 SNS 분석에 적합한 Twitter 클래스 이용
 - morphs() 메서드 이용해 입력 문자로 들어온 문장을 형태소 단어 형태로 토큰화해 list 객체로 반환
 - 문장을 형태소 단어 형태로 반환하는 별도의 tw_tokenizer() 함수 생성
- 사이킷런 TfidfVectorizer 이용해 TF-IDF 모델 생성
 - tokenizer 는 위에서 만든 함수 이용
 - ngram 은 (1,2), min_df 는 3, max_df 는 상위 90%로 제한
- 로지스틱 회귀 이용해 분류 기반의 감성 분석 수행
 - 하이퍼 파라미터 C의 최적화 위해 GridSearchCV 이용
- 데이터셋 이용해 최종 감성 분석 예측 수행
 - 학습 데이터에 사용된 TfidfVectorizer 객체 변수 tfidf_vect 이용해 transform() 을 테스트 데이터의 document 칼럼에 수행